LECTURE 6.1

Loss Functions and Gradients

You have spent five lectures building models and none of them fitted. Today: where the loss comes from, how a parameter moves, and four slides at the end that make the second half of this course possible.

Remus Teodorescu

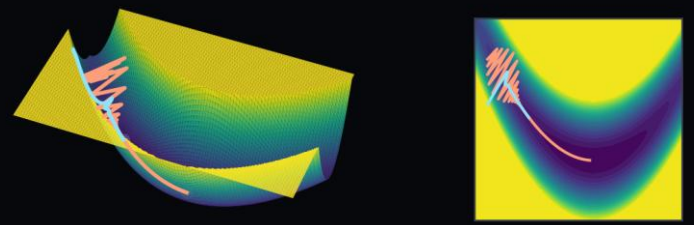
With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

likelihood · descent · Adam · backpropagation · the bridge

THE LOOP YOU HAVE ALREADY WRITTEN



Four lines in Ex_02 notebook 03, and you ran them without being told what any of them did. Today is the inside of boxes two, three and four.

the loss	derived, not memorised
the optimiser	SGD, momentum, Adam
the gradients	backpropagation, once

LEARNING OBJECTIVES

What You Will Be Able To Do

THE GOALS, IN PLAIN WORDS

- **derive** — squared error and cross entropy from one recipe, rather than recalling them
- **say** — what distribution your loss assumes, for any loss you use
- **explain** — why momentum and Adam exist, in terms of the surface they walk
- **state** — what backpropagation computes, and why it costs one extra pass
- **differentiate** — a network with respect to its input, and say why that matters
- **write** — a loss term containing no data at all

WHAT TODAY ASSUMES

from L4.2	capacity, and the loss surface
from L5	the architectures the loss is applied to
not needed	any statistics beyond a probability density

THE RECIPE IS THE POINT

Choose a distribution, write the likelihood, take the log, negate. Every loss in this course is that, and the choice of distribution is where your engineering judgement sits.

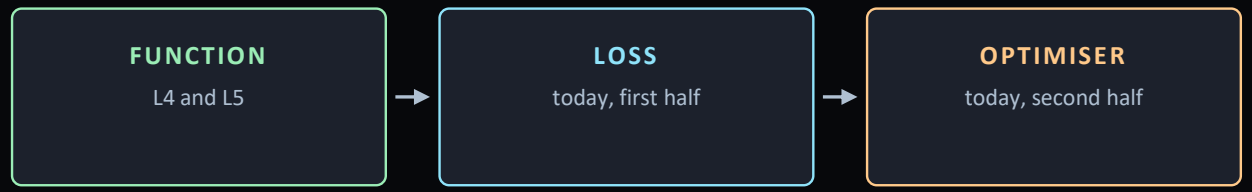
AND SLIDE 22 OPENS PART 2

A loss term need contain no data at all. Everything from L7 onward rests on that one slide.

The Third Ingredient

A PARAMETERISED FUNCTION, A LOSS, AN OPTIMISER

- **L3.1 said three ingredients** — a function, a loss, an optimiser
- **L4 and L5 gave you the function** — design it, count it, choose its activation
- **today closes the other two** — and the loss goes first
- because that is where the modelling assumptions live
- **then four slides that look like an appendix** — and are the reason this lecture exists



TODAY, IN FIVE PARTS

3 – 8	the loss, from maximum likelihood
9 – 14	descent, SGD, momentum, Adam
15 – 17	backpropagation, on a toy example
18 – 19	initialisation, and what it prevents
20 – 23	the bridge into Part 2

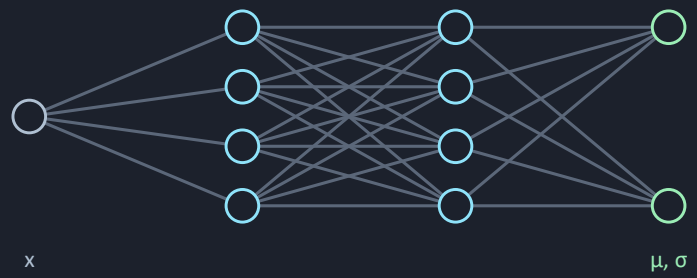
Slides 20 to 23 are four ideas, one slide each, that Part 2 assumes you already hold. They cost six minutes here and they save an entire exercise session in week seven. If we run short of time, something in the middle goes, not those.

A Loss Is Not Chosen From a List

THE QUESTION NOBODY ASKS OUT LOUD

- **most courses hand you a table** — squared error here, cross entropy there
- **there is one recipe underneath** — and it comes from statistics, not machine learning
- stop thinking of the network as predicting a number
- **think of it as predicting a distribution** — and choose the parameters that make your data likely
- **that is maximum likelihood** — a hundred years old, and you have used it

WHAT THE NETWORK ACTUALLY EMITS



The output layer does not give you the answer. It gives you the parameters of a distribution over answers, and the answer is that distribution's most likely value.

a real number	Gaussian → mean
yes or no	Bernoulli → probability
one of K	categorical → K probabilities

The Recipe, in Five Steps

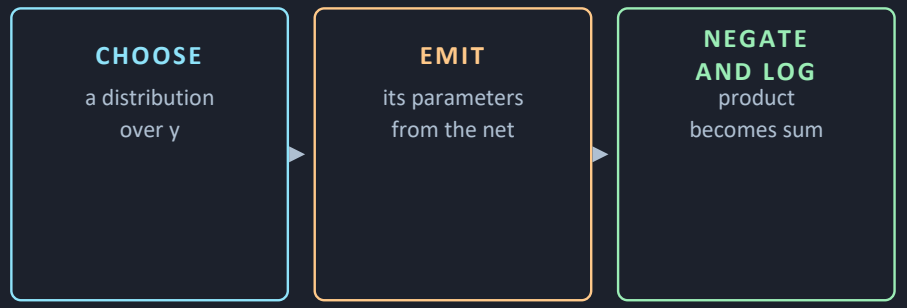
APPLY THIS AND THE LOSS FALLS OUT

- **choose a distribution** — that matches what you are measuring
- have the network emit its parameters
- write the likelihood of your data, take the log, negate it
- **the negation is convention** — optimisers minimise
- **step one is the only one where judgement enters** — the rest is algebra

Every loss on the next three slides is this line with a different distribution substituted in.

$$\hat{\theta} = \text{argmin}_{\theta} \; -\sum_{i} \log P\big(y_i \,|\, f(x_i, \theta)\big)$$

THE MACHINE



WHY THIS IS WORTH SIX MINUTES OF YOUR LIFE

When your measurement is not Gaussian — a count, a lifetime, a heavy-tailed sensor with occasional spikes — the table has no row for you. The recipe does. You write down the distribution you believe, and it hands you the loss.

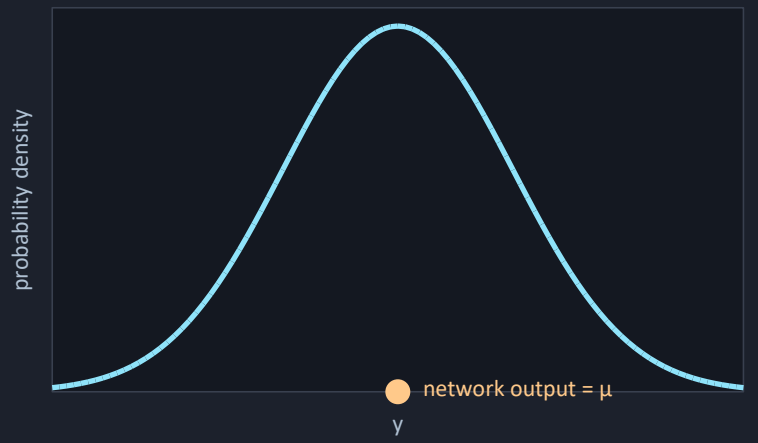
A Real Number → Squared Error

THE CASE YOU HAVE BEEN USING WITHOUT NOTICING

- you measure a temperature, a stress, a voltage
- assume the measurement is truth plus independent Gaussian noise
- **substitute** — the exponential cancels against the logarithm
- and what is left is the sum of squared errors
- **so it is not a convenient choice** — it is what Gaussian noise implies

$$-\log P = \frac{1}{2\sigma^2}\sum_i \big(y_i - f(x_i,\theta)\big)^2 + \text{const}$$

ONE MEASUREMENT, AS A DISTRIBUTION



The network places this bell over each input. Maximising the height it assigns to your actual measurements is the same thing as minimising the squared distance from their centres.

Ex_06 notebook 01 evaluates both surfaces on a 601 × 601 grid and finds the same minimiser to three decimals.

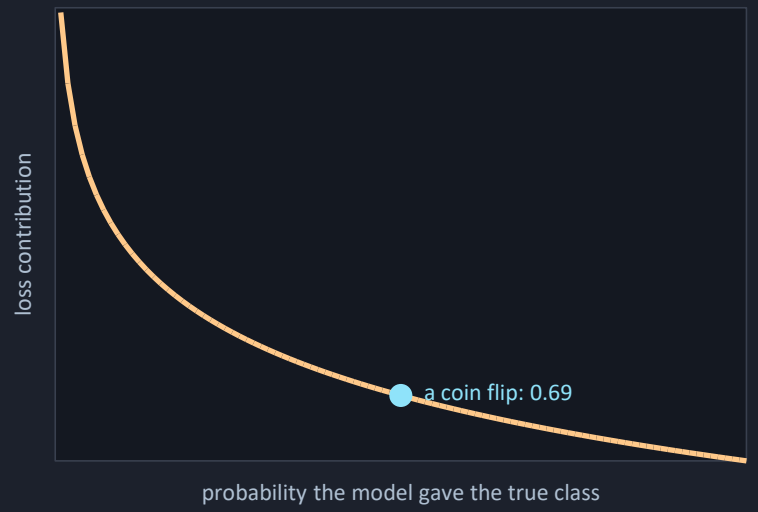
Yes or No → Binary Cross Entropy

THE SAME RECIPE, A DIFFERENT DISTRIBUTION

- **now the output is a class** — the bearing failed or it did not
- **the distribution is Bernoulli** — one number, the probability of failure
- **a network outputs any real number** — so squash it with a sigmoid
- substitute, and you get binary cross entropy
- **read it as an accountant** — confident and wrong is charged heavily

$$\mathcal{L} = -\sum_{i}[y_i \log p_i + (1 - y_i)\log(1 - p_i)]$$

THE PENALTY FOR BEING CONFIDENTLY WRONG



The curve goes to infinity at zero. A model that assigns literally no probability to something that then happens receives an infinite penalty, and in floating point that is a NaN in your training log.

One of K → Cross Entropy

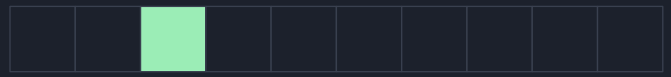
THE GENERAL CLASSIFICATION CASE

- **with K classes the distribution is categorical** — K probabilities summing to one
- softmax produces them
- the loss is the negative log of the probability given to the correct class
- **PyTorch's CrossEntropyLoss applies the softmax itself** — do not apply it twice
- **and the check worth memorising** — an untrained K-class model should report log K

$$p_k = \frac{\exp(z_k)}{\sum_j \exp(z_j)}$$

LOGITS → PROBABILITIES

$$\mathcal{L} = -\sum_i \log p_{y_i}$$



one-hot target: only the true class contributes

Ex_06 notebook 01 writes softmax and cross entropy in NumPy and matches PyTorch to zero. Doing that once removes all the mystery.

HONESTY

What the Recipe Costs You When You Are Wrong

THE ASSUMPTION IS NOT DECORATIVE

Squared error is optimal for Gaussian noise. Real instruments produce occasional wild readings — a dropped sample, an electrical transient, a loose thermocouple — and a Gaussian says those are essentially impossible.

WHAT ONE BAD READING DOES

Because a Gaussian assigns almost no probability to a far-out value, the only way for the fit to explain one is to move. So the model drags itself towards a single corrupt point, and the squared penalty makes that drag grow with the square of the distance.

In Ex_06 notebook 01 one bad reading in forty moves the least-squares slope by nine per cent and the intercept by thirty-five. That is one measurement out of forty, and it is not a pathological one.

The fit is still the maximum likelihood answer. It is the right answer to the wrong question.

WHAT THE RECIPE OFFERS INSTEAD

Assume a heavier-tailed distribution — a Laplace, which says large deviations are unlikely but not absurd. Run the same five steps.

Out comes the sum of absolute errors rather than squared errors. The absolute-error fit on that same corrupted dataset lands within two per cent of the truth on both parameters.

You did not look up a robust loss. You changed one modelling assumption and the recipe produced the loss that follows from it. That is the entire argument for teaching it this way.

The cost is honest too: absolute error has no derivative at zero and optimises less smoothly.

Choosing a loss is choosing a noise model. Say which one you assumed, in the report, in one sentence.

Gradient Descent, and the Surface It Walks

DOWNHILL, IN SMALL STEPS, FOR EVER

- **the loss is a surface over parameter space** — one dimension per parameter
- **you cannot see it** — but you can compute its slope where you stand
- **step downhill, repeat** — that is gradient descent
- steps are long where it is steep and short near the bottom, automatically
- **and it is local** — it cannot know whether a better basin exists elsewhere

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathcal{L}(\theta_t)$$

ONE PARAMETER, ONE SURFACE

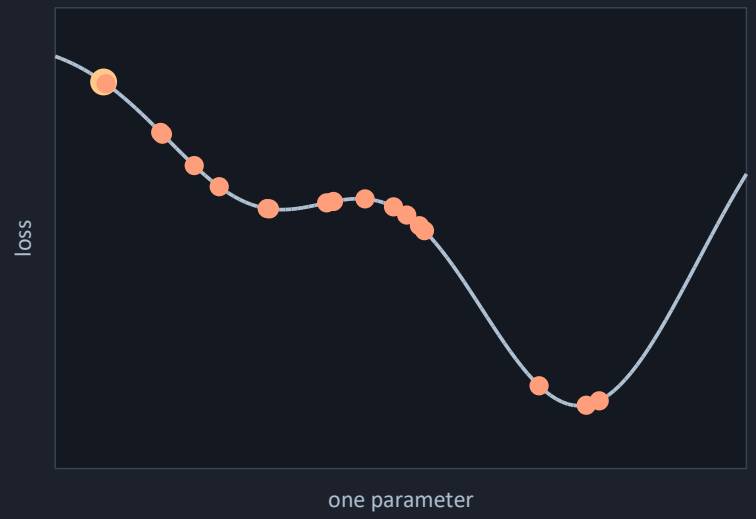
Nine steps from the amber start. Long strides on the steep flank, short ones near the bottom — and it settles in the basin it happened to be nearest, not the deepest one on the slide.

Stochastic Gradient Descent — Noise as a Feature

TWO REASONS, AND THE SECOND IS THE INTERESTING ONE

- **the full gradient is a sum over every sample** — expensive on a large dataset
- **so use a minibatch** — a noisy estimate of the same thing
- that is the obvious reason, and it is not the interesting one
- **the noise itself helps** — a noisy step can leave a shallow basin
- so batch size is not only a memory setting

THE SAME SURFACE, WITH NOISE



The noisy walk does not stop in the first dip. It rattles across it and finds the deeper basin — which is luck rather than intelligence, but it is luck you get for free on every run.

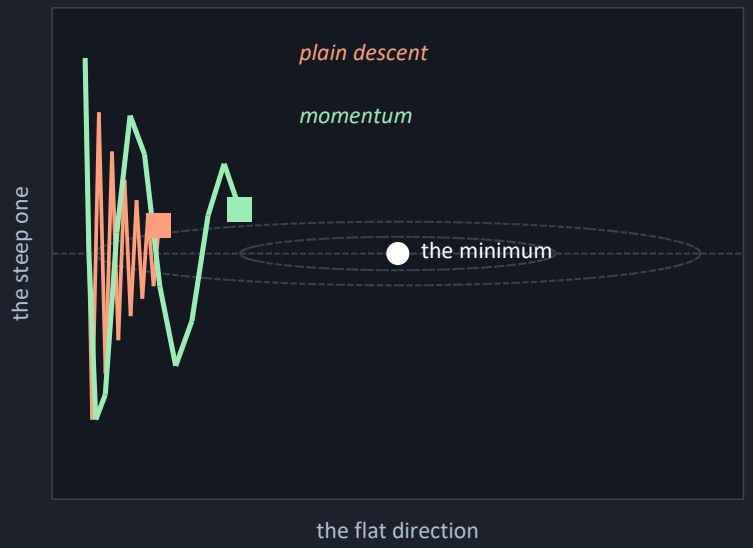
Momentum — Averaging the Zig-Zag Away

THE PROBLEM MOMENTUM SOLVES, DRAWN ON THE RIGHT

- most loss surfaces have directions far steeper than others
- in a long narrow valley the gradient points across it, not along
- **momentum averages recent gradients** — the across-valley components cancel
- **the picture in the name is right** — a ball with mass, not a point
- **Ex_06 notebook 02 measures it** — same rate, same data, a large difference

$$m_{t+1} = \beta m_t + (1-\beta)\nabla_\theta \mathcal{L} \quad \theta_{t+1} = \theta_t - \alpha\, m_{t+1}$$

A NARROW VALLEY, SEEN FROM ABOVE



Dashed rings are contours of equal loss; the squares mark where each method stands after twelve steps. Momentum has covered more than twice the distance along the valley, with no change to the learning rate.

Adam — A Learning Rate Per Parameter

TWO RUNNING AVERAGES, AND A DIVISION

- **momentum fixed the direction** — Adam fixes the scale
- and it does so per parameter
- **two running averages** — the mean of gradients, and the mean of their squares
- **divide one by the root of the other** — small consistent gradients still move
- which is why Adam is the default, and why almost nobody tunes its betas

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

WHAT EACH PIECE BUYS

gradient	the direction downhill
+ momentum	averages out the zig-zag
+ squared average	per-parameter scaling
+ bias correction	fixes the first few steps
= Adam	the default since 2015

THE SETTINGS TO USE

lr = 1e-3, betas = (0.9, 0.999), eps = 1e-8. Change the first. Leave the rest alone unless you have a measurement that says otherwise.

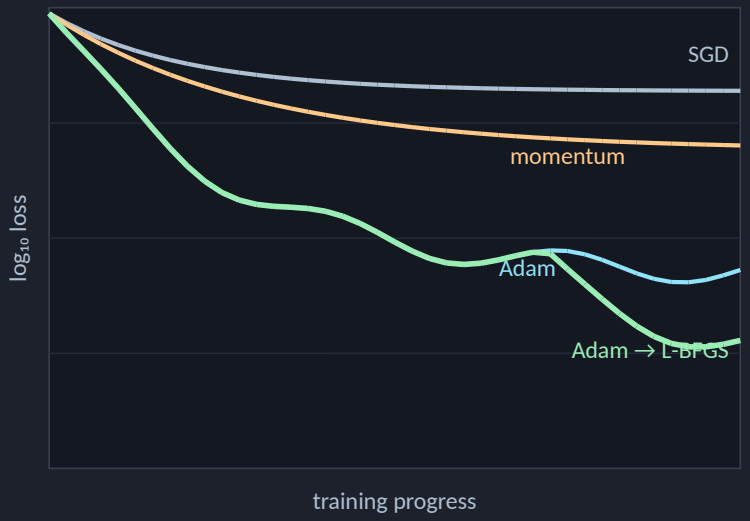
EVIDENCE

Four Optimisers, One Problem, Measured

SAME MODEL, SAME DATA, SAME SEED

- **321 parameters, a damped structural response** — only the optimiser changes
- plain SGD stalls at 3.5e-02
- **momentum, same learning rate** — 3.4e-03
- **Adam** — 2.2e-05
- **Adam then L-BFGS** — 1.0e-06, and the caveat is on the right

FINAL LOSS, LOWER IS BETTER



Shape from the measured runs; the ordering and the endpoints are the real numbers. Note the ripple on the cyan curve — that is Adam being non-monotone.

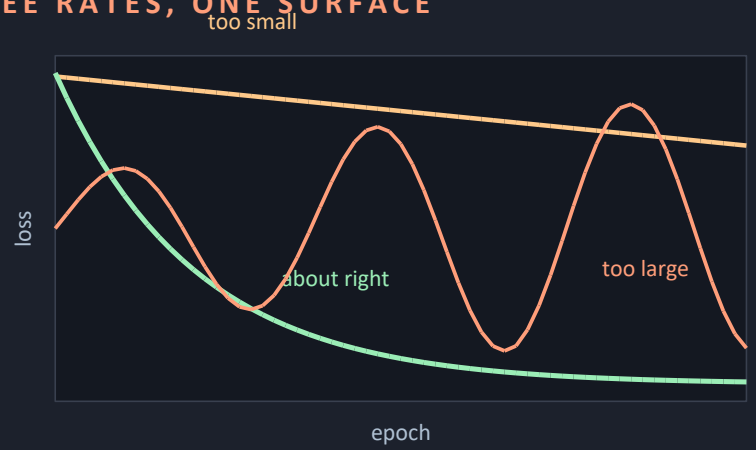
PRACTICE

The Learning Rate, and Everything Else

ONE HYPERPARAMETER MATTERS MUCH MORE THAN THE REST

- tune one number, tune the learning rate
- **too large and it diverges** — visibly, which is the easy failure
- **too small and it falls slowly for ever** — and looks like a working run
- search on a log scale, in factors of ten
- **and a decay schedule gives you both** — travel early, settle late

THREE RATES, ONE SURFACE



WHAT TO TUNE, IN ORDER

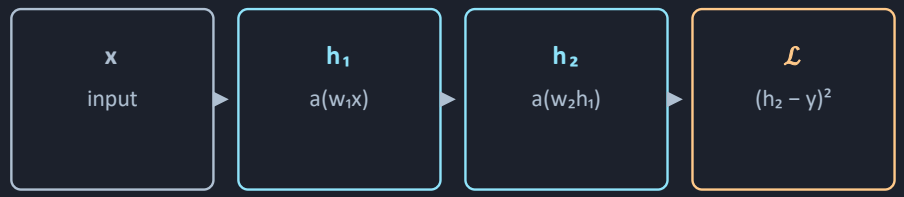
1	learning rate — decades, not decimals
2	how long you train
3	network size — L4.2 slide 19

A Network Small Enough To Do By Hand

FORWARD FIRST, AND STORE EVERYTHING

- **every optimiser needs the same thing** — the gradient on every parameter
- **backpropagation gets all of them** — for about one extra forward pass
- the forward pass computes and stores every intermediate value
- that storage is why training needs several times the memory of inference
- and why batch size is limited by memory rather than by mathematics

THE FORWARD PASS



Two weights, two hidden values, one loss. Everything on the next two slides is this picture read from right to left.

WHAT IS STILL IN MEMORY AFTERWARDS

x, h_1, h_2	every intermediate value
the graph	which operation produced each
and only then	the backward pass can start

Training a network needs several times the memory of running one, and this is exactly why. Reducing the batch size is the first thing to try when a GPU runs out of memory.

The Chain Rule, Applied Backwards

ONE SWEEP, RIGHT TO LEFT

- **start at the loss** — where the derivative is immediate
- **walk backwards** — at each node, multiply what arrived by that node's own derivative
- **that is the whole algorithm** — the chain rule, in the order that permits reuse
- **and the reuse is the trick** — every parameter shares the same partial products
- so all of them cost about what one costs

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial w_1}$$

THE BACKWARD PASS

$\partial \mathcal{L}/\partial w_1$	$\partial \mathcal{L}/\partial h_1$	$\partial \mathcal{L}/\partial h_2$	$\partial \mathcal{L}/\partial \mathcal{L} = 1$
the answer	arrives from the right	one step back	start here

Read this row right to left — the arrows point the way the numbers travel in the forward pass, and the derivatives travel the other way.

local derivative	cheap — you stored the values
incoming derivative	reused by everything upstream
their product	this weight's gradient

Ex_02 notebook 02 does this by hand and checks it against autograd.

Solved, and Therefore Automatic

WHAT THE FRAMEWORK IS DOING WHILE YOU WATCH

- **you will never implement it** — PyTorch records every operation on a tensor
- three things are worth knowing anyway

- **the graph is built during the forward pass** — so Python control flow is recorded as taken
- **calling backward frees it** — call it twice and you get an error, not a wrong answer
- **and nothing in that description mentions weights** — which is slide 20

THE THREE THAT BITE

zero_grad()	gradients add up if you forget
backward() once	the graph is freed after it
no_grad()	for inference — saves the memory
detach()	cuts the graph deliberately

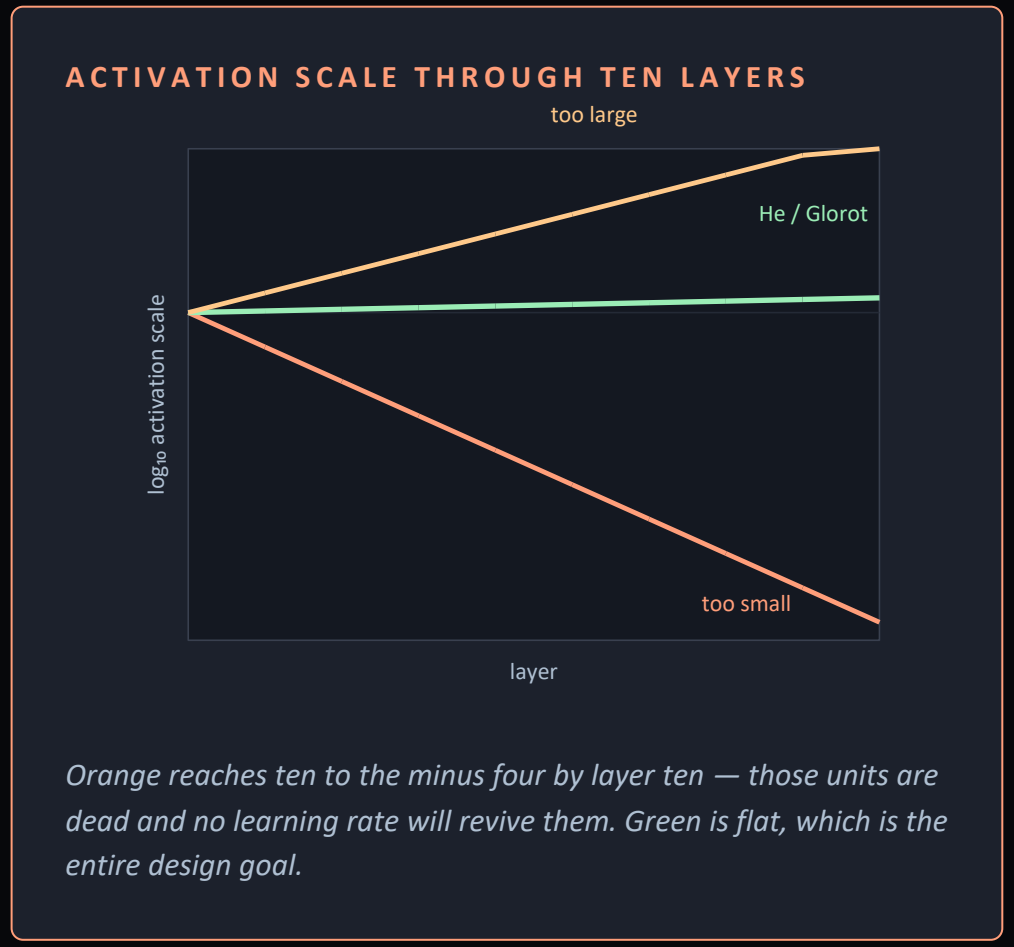
THE SENTENCE THAT MATTERS IN THREE SLIDES' TIME

Autograd does not know what a weight is. It differentiates a tensor with respect to any other tensor it can reach through the recorded graph — and the input is a tensor like any other.

Where the Parameters Start

TWO FAILURES, BOTH BEFORE THE FIRST STEP

- **set every weight to zero** — and every unit computes the same thing, for ever
- so start random. But with what spread?
- **a little too large and activations grow layer by layer** — a little too small and they vanish
- **the same happens to gradients on the way back** — L4.2, before any training
- **the plot is a ten-layer network at three scales, untrained** — the scale alone does this



Orange reaches ten to the minus four by layer ten — those units are dead and no learning rate will revive them. Green is flat, which is the entire design goal.

He and Glorot — One Line, Two Rules

PICK THE VARIANCE THAT KEEPS THE SCALE CONSTANT

- **a unit sums its inputs** — variance grows with the number of them
- so scale the initial weights by one over that count
- **Glorot balances forward and backward** — using the average of fan-in and fan-out
- **He corrects for ReLU discarding half its input** — a factor of two
- **PyTorch does this by default** — it matters when you write a layer yourself

He, on the left, for ReLU. Glorot, on the right, for tanh.

$$\mathrm{Var}(w) = \frac{2}{D_{\text{in}}} \qquad \mathrm{Var}(w) = \frac{2}{D_{\text{in}} + D_{\text{out}}}$$

WHICH ONE, AND WHEN

ReLU networks	He — variance 2 / fan-in
tanh, sigmoid	Glorot — 2 / (fan-in + fan-out)
every PINN in Part 2	Glorot, because tanh
biases	zero is fine, and is the default
a custom layer	the only time you must act

THE HABIT WORTH FORMING

Print the mean and standard deviation of each layer's activations on the first batch. Two lines, once, and it catches a dead network before you have spent an hour wondering why the loss will not move.

Autograd Differentiates With Respect To Anything

THE ONE CALL THE WHOLE OF PART 2 RESTS ON

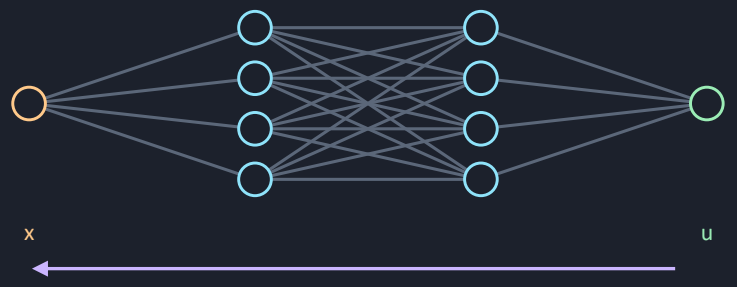
torch.autograd.grad(u, x) — the derivative of the network's output with respect to its input. Not its weights. Its input.

WHY THIS IS NOT A TRICK

- backpropagation walks a graph of tensors
- and that graph does not know which tensors are weights
- so you can ask for the derivative with respect to an input
- **for a network u of position x** — that gives du/dx, and again for the second derivative
- **you have already done this** — Ex_02 notebook 02, on an analytic function

$$\frac{\partial u}{\partial x}, \frac{\partial^2 u}{\partial x^2}$$

GRADIENTS GOING THE OTHER WAY



∂u/∂x — the same backward sweep, one node further left

w.r.t. weights	training — what you know
w.r.t. inputs	physics — what is next
twice	second derivatives, for a PDE

L-BFGS, and Why PINNs Run Two Optimisers

A DIFFERENT KIND OF OPTIMISER, AND WHEN TO SWITCH

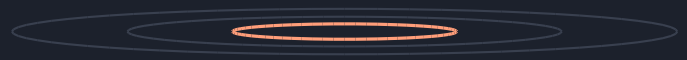
- everything so far used only the slope

- **a second-order method uses curvature too** — and near a minimum that is much better
- L-BFGS approximates the curvature from the gradients it has already seen

- **it is full-batch** — so it needs the same loss every call, which rules out minibatches
- **a physics residual is badly conditioned** — Adam gets close, L-BFGS finishes

A BADLY CONDITIONED BOWL

curvature ratio ≈ 50



THE RECIPE, AND ITS MEASURED VALUE

Adam, 2000 steps	into the right basin
then L-BFGS, 60	down to 1.0e-06
worth	264 × over Adam alone

A Loss Term Need Contain No Data At All

THE CONCEPTUAL JUMP OF THE ENTIRE SECOND COURSE

A loss is any differentiable number you want driven to zero. Nothing in that sentence requires a measurement, a label, or a target.

SO WRITE THE EQUATION AS THE LOSS

- **take the heat equation** — move everything to one side
- **what is left is a residual** — zero when the equation is satisfied
- **let the network be that function** — slide 20 gives you every derivative in it
- **square the residual at sampled points and average** — that is a loss with no data in it
- and every technique in this lecture applies to it unchanged

TWO LOSSES, SIDE BY SIDE

SUPERVISED	RESIDUAL
needs x, needs y, compares them	needs x only, compares to physics

$$\mathcal{L}_{\text{phys}} = \frac{1}{N}\sum_{j}\left(\frac{\partial u}{\partial t} - \alpha\frac{\partial^2 u}{\partial x^2}\right)^2$$

No y appears anywhere in that expression. The only inputs are the points at which you chose to evaluate it — chosen rather than measured, and free to resample every epoch. What it costs is that you have nothing to hold back.

Two Terms, and the Number Between Them

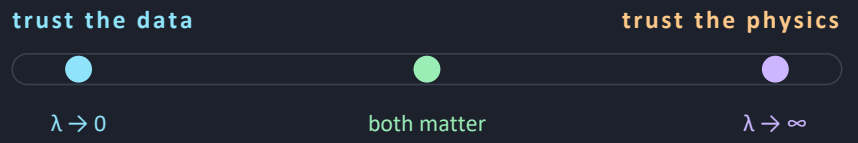
λ IS NOT A HYPERPARAMETER TO TUNE AWAY

- **in practice you have both** — sparse measurements and a governing equation
- add the two losses, with a number between them

- **the temptation is to sweep lambda and report the best** — that is fitting to the test set
- **lambda is the ratio of the two noise scales** — go back to slide 5, where sigma dropped out
- **so it is measurable** — your thermocouple has a stated accuracy

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda \mathcal{L}_{\text{phys}}$$

WHAT λ IS SAYING



At the left the equation is ignored and you have a curve fit. At the right the measurements are ignored and you have a solver. Every useful problem is between them, and where you sit is a claim about your instruments.

WHAT A REPORT MUST CONTAIN

The value of λ, the argument for it in one sentence, and what changed when you moved it by a factor of ten. Three lines. Every Part 2 report is marked on them.

Failure Modes of Training

THE LOSS IS NaN AFTER FOUR STEPS

Learning rate too large, or a log of zero in a cross entropy, or an exploding initialisation. Divide the learning rate by ten first — it is one keystroke and it is the cause most of the time.

THE LOSS FALLS, SLOWLY, FOR EVER

Learning rate too small, or dead units from a bad initialisation. It looks like a working run, which is what makes it expensive. Print the activation statistics on the first batch.

THE LOSS DOES NOT MOVE AT ALL

Almost always the loop rather than the model: a missing zero_grad, an optimiser built over the wrong parameters, or a detach that cut the graph. Check that the gradients are non-zero before you touch the architecture.

WHAT TO CHECK, IN ORDER OF COST

free	divide the learning rate by ten
free	check the first loss ≈ ln K
free	print gradient norms after backward
cheap	activation stats on batch one
cheap	overfit five samples
real work	reconsider the noise model

The last row is slide 8. If your loss assumes Gaussian noise and your sensor produces occasional spikes, no amount of optimiser tuning will fix a fit that is being dragged by three bad readings.

YOUR TURN

Ex_06 — Notebooks 01 and 02

DERIVE THE LOSS, THEN RACE THE OPTIMISERS

- **notebook 01** — the recipe with the arithmetic done, by hand then in PyTorch
- **notebook 02** — four optimisers on one problem, and the figure on slide 13 is yours to reproduce
- **both are short** — and notebook 02 is the one to run twice
- the report asks which optimiser you would use, and why

THE NOTEBOOKS

00	environment check — read only
01	losses, derived and measured
02	SGD · momentum · Adam · L-BFGS
03	transfer, then fine-tune — L6.2
04	quantisation — L6.2
05	the report

THE MARKED QUESTION

Somebody asks why you are running two optimisers instead of one. Answer them, with a number from notebook 02 and the condition under which your answer stops holding.

SUMMARY

Key Takeaways

1. **A loss is a noise model**

   Choose the distribution you believe, negate its log likelihood, and the loss follows. MSE is the Gaussian answer and it is wrong when your sensor spikes.

2. **The learning rate is the hyperparameter**

   Search it in decades, not decimals. Everything else on the optimiser slides matters less than getting this within a factor of ten.

3. **Adam is the default, and is not monotone**

   Report which loss you are quoting — the final one or the best one along the run. They differed by two orders of magnitude in Ex_06.

4. **Backpropagation is the chain rule, reused**

   All the gradients cost about one extra forward pass, and the graph it walks connects tensors — not weights.

5. **λ is a statement, not a knob**

   It is the ratio of how much you trust your physics to how much you trust your instruments, and it belongs in a report as a number with a justification.

Next — L6.2: what happens to a model after it has been trained, and the one lecture in Part 1 with no equations in it.

OVER TO YOU

Questions

TEN TO TEST WHETHER TODAY LANDED

- **what distribution does squared error assume** — and when is that assumption wrong?
- **why is the negative log taken** — and which part is convention?
- an untrained ten-class model reports what loss, and why?
- why does plain gradient descent zig-zag in a narrow valley?
- what two averages does Adam keep, and what does each fix?
- why does backpropagation cost about one extra forward pass?

AND FOUR MORE

seven	why does training need more memory than inference?
eight	why can autograd differentiate with respect to an input?
nine	why do PINNs run Adam and then L-BFGS?
ten	what is the weight between two loss terms, physically?

THE ONE WITH NO SETTLED ANSWER

You have no measurements at all. What is your loss, and how would you know it had converged?

BEFORE L6.2

Run notebooks 01 and 02. Reproduce the four-optimiser figure.

SOURCES

References and Further Reading

THE TWO BOOKS, AND WHERE THEY DIVIDE

Prince, Understanding Deep Learning — chapter 5 for loss functions from maximum likelihood, which is the best short treatment of the recipe on slide 4. Chapter 6 for fitting: gradient descent, stochastic gradient descent, momentum and Adam. Chapter 7 for gradients and initialisation.

Goodfellow, Bengio & Courville — chapter 8.6 for quasi-Newton methods and L-BFGS, which Prince does not cover at all because chapter 6 stops at Adam. Chapter 4.2 for numerical conditioning, which is the reason the Adam-to-L-BFGS handoff exists rather than being a superstition.

AND ONE OUTSIDE THEM, FOR PART 2

Liu (2025), chapter 2.7.5 – 2.7.6, for the two-stage optimiser implementation that every Part 2 exercise uses. This is an implementation convention rather than a result, and it is the only place in this lecture sourced outside the two books.

Kingma & Ba (2015), Adam: A Method for Stochastic Optimization — nine pages, and the algorithm box on page two is the whole of slide 12.

READ IN THIS ORDER

before Ex_06 nb 01	UDL ch. 5.1 – 5.3
before Ex_06 nb 02	UDL ch. 6.1 – 6.4
if slide 21 was fast	GBC ch. 8.6, then ch. 4.2
before L6.2	UDL ch. 9.3
before L7.1	slides 20 to 23 again

NEXT

L6.2 — Post-Training and Reinforcement Learning

Training a model from scratch is now the unusual case. Next time: starting from somebody else's weights, what post-training means for a 2026 model, reinforcement learning properly, and how to make a network run fast enough to steer a car.

BEFORE THE NEXT SESSION

read	UDL ch. 9.3 — transfer learning
run	Ex_06 notebooks 01 and 02
bring	your four-optimiser loss plot
think about	what λ would be for your own project

Bring the plot. The gap between the Adam curve and the Adam-then-L-BFGS curve is the single most useful figure you will produce this week, and it is the one you will redraw in every Part 2 exercise.